

Transaction-level approval enforcement on Besu

Proof of concept, 15 September 2026 · unmodified Besu 26.8.1 · measured on one laptop (Apple M2 Max)

It works as a plain Besu plugin — no fork, no patch, one JAR. The node refuses to include any transaction whose actual execution differs from what was authorized: the block producer compares a fingerprint of the execution it just performed against a signed approval and drops the transaction *before* the state change is committed. Not "it fails with a receipt" — it never enters the block. **Throughput cost: about 7 %.** The remaining ceiling belongs to Besu itself, not to the gate.

MEASURED THROUGHPUT — 60 S ADAPTIVE LOAD, ETH TRANSFERS, 10 WALLETS, 1 S BLOCKS

Configuration	Avg tx/s	Confirmed	Failed	Block fill
Straight to Besu — neither the proxy nor the gate in the path	765	45,887	0	15.9 %
Through the authorization proxy, gate off	727	43,622	0	—
Through the authorization proxy, gate on	671	40,198	0	—

Read it this way: the gate costs ~7 % and the proxy's own authorization work ~5 %. Taking both out entirely buys only 13 %, and at that ceiling Besu's blocks are just **15.9 % full** — so what runs out first is Besu's per-transaction RPC and pool handling, not block space or our enforcement. Zero failures in every run; all 40,198 confirmations were re-verified against on-chain receipts.

WHERE THE TIME GOES

- **Inside the block producer: 7–22 µs per transaction** for the plugin's own work (approval lookup, fingerprint, comparison). Negligible.
- **Per client request: 17.5 ms → 21.1 ms** end-to-end through the proxy. The ~3.5 ms the gate adds is a pre-flight simulation on the node plus a signature.
- Confirmation latency ~1.0 s median in every configuration — that is the 1 s block cadence, unchanged by the gate.

WHAT WAS VERIFIED

- 20 end-to-end scenarios / 22 checks green on a real node, including **contract deployment**.
- **Coexists with the target stack's own sequencer plugin** — both plugins, both tracers, one node; our gate still decides.
- The headline case: an approved call overtaken in the same block by a higher-fee transaction that redirects it is **excluded** — no receipt, nonce and balances untouched.

LIMITS WORTH KNOWING BEFORE ANYONE PLANS AROUND THIS

- **Enforcement is producer-only.** It is a block-building rule, not a consensus rule: every producer must run the plugin, and a block received from elsewhere is not re-checked. Making it a consensus rule would require changing Besu, not a plugin.
- **Deployments and other state-binding transactions are approved against exact state.** A concurrent transaction in the same block touching one of the same storage slots invalidates the approval; the client must re-submit after a fresh pre-flight. Ordinary calls use a looser binding and are unaffected.
- **An expired approval is dropped at the next block-building round**, not on a timer — Besu gives plugins no way to remove a transaction from the pool. In the meantime it simply cannot be selected.
- **Version coupling.** The Besu plugin interfaces used here are marked unstable and were split into separate artifacts in 26.8.x. Pin the node version and re-run the suite on every upgrade. Requires JDK 25.
- **Co-running with the target stack's ZK tracer requires an Osaka-fork chain;** on older forks that tracer refuses and block building stops. This is their constraint, not ours, but it pins the network's fork.
- **Reorgs need an answer.** Transactions returned to the pool by a reorg wait for a fresh approval, which the proxy does not currently re-send. Approvals also cannot be revoked once issued (they expire).
- **Not yet covered:** EIP-7702 and blob transactions are refused outright; bundle-style plugins that commit transactions themselves must be checked before coexisting.
- **All numbers come from one laptop** running the load generator, the proxy, PostgreSQL, Redis and Besu together. They bound this machine, not the design — a production sizing exercise is still owed.

Evidence: dashboard screenshots, per-run history records, status samples and gzipped receipt lists for all three configurations, plus the scenario suite and the reproduction commands, live with the PoC branch (`poc/ops-besu-signed-approvals`, see `DEMO.md`). Nothing here is simulated: the runs use the real authorization service, its own database and signing keys, and a real Besu node.